

Lab#7 – White-box testing

วัตถุประสงค์การเรียนรู้

1. ผู้เรียนสามารถออกแบบการทดสอบแบบ White-box testing ได้
2. ผู้เรียนสามารถวิเคราะห์ปัญหาด้วย Control flow graph ได้
3. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Line coverage ได้
4. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Block coverage ได้
5. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Branch coverage ได้
6. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Condition coverage ได้
7. ผู้เรียนสามารถออกแบบกรณีทดสอบโดยคำนึงถึง Branch and Condition coverage ได้

โจทย์: CLUMP COUNTS

Clump counts (<https://codingbat.com/prob/p193817>) เป็นโปรแกรมที่ใช้ในการนับการเกาะกลุ่มกันของข้อมูลภายใน Array โดยการเกาะกลุ่มกันจะนับสมาชิกใน Array ที่อยู่ติดกันและมีค่าเดียวกันตั้งแต่สองตัวขึ้นไปเป็นหนึ่งกลุ่ม เช่น

[1, 2, 2, 3, 4, 4] → 2

[1, 1, 2, 1, 1] → 2

[1, 1, 1, 1, 1] → 1

ซอร์สโค้ดที่เขียนขึ้นเพื่อนับจำนวนกลุ่มของข้อมูลที่เกาะอยู่ด้วยกันอยู่ที่

<https://github.com/ChitsuthaCSKKU/SQA/tree/2025/Assignment/Lab7>

โดยที่ nums เป็น Array ที่ใช้ในการสนับสนุนการนับกลุ่มของข้อมูล (Clump) ทำให้ nums เป็น Array ที่จะต้องไม่มีค่าเป็น Null และมีความยาวมากกว่า 0 เสมอ หาก nums ไม่เป็นไปตามเงื่อนไขที่กำหนดนี้ โปรแกรมจะ return ค่า 0 แทนการ return จำนวนกลุ่มของข้อมูล

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

แบบฝึกปฏิบัติที่ 7.1 CONTROL FLOW GRAPH

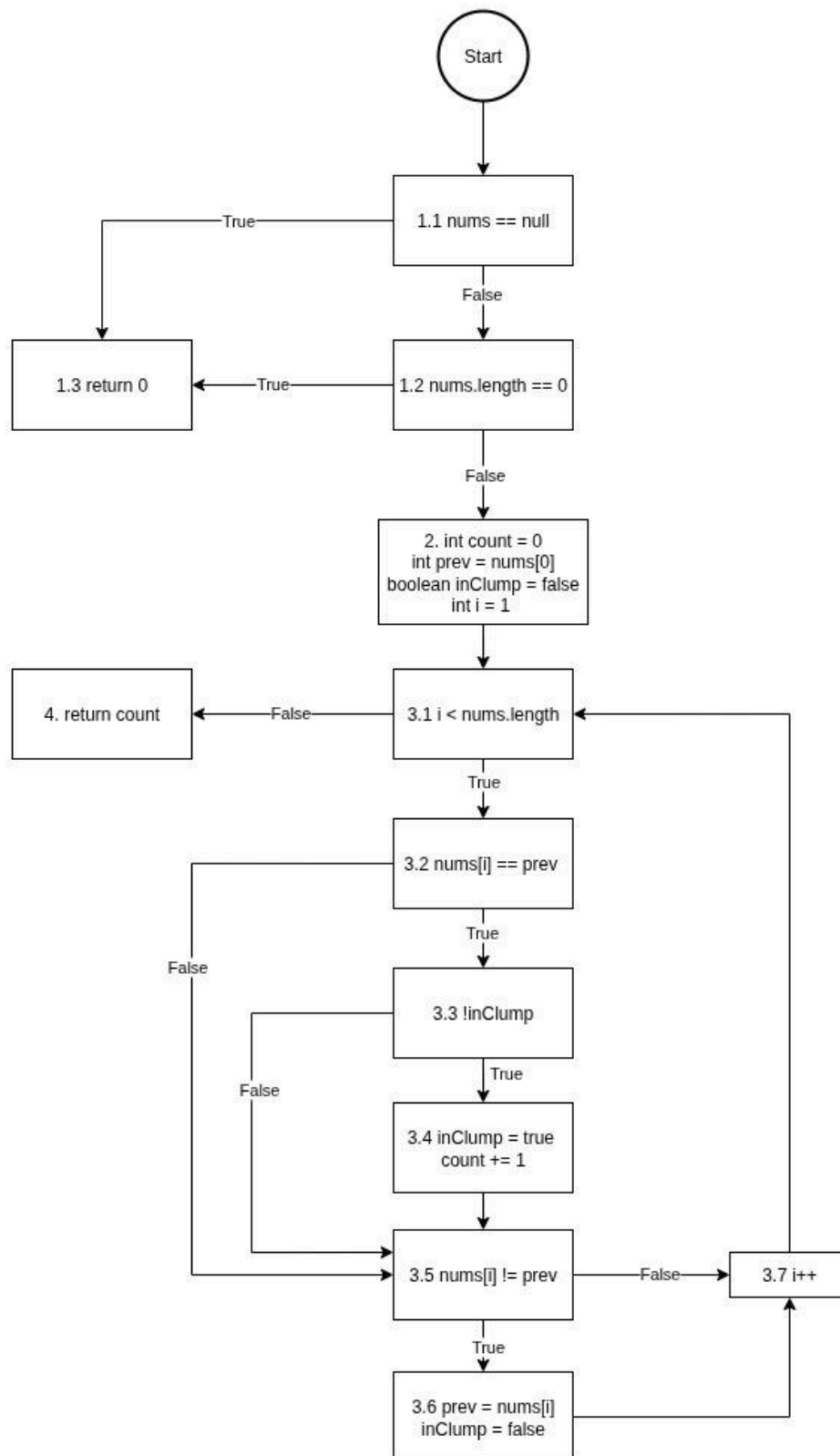
จากโจทย์และ Source code ที่กำหนดให้ (CountWordClumps.java) ให้เขียน Control Flow Graph (CFG) ของเมธอด countClumps() จากนั้นให้ระบุ Branch และ Condition ทั้งหมดที่พบใน CFG ให้ครบถ้วน

ตอบ

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction



CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

Branch:

B1 = 1.1

เช็ค `nums == null` ถ้า true จะ return 0 แต่ถ้า false จะเช็คเงื่อนไขถัดไป

B2 = 1.2

เช็ค `nums.length == 0` ถ้า true จะ return 0 แต่ถ้า false จะทำการประกาศตัวแปรที่จำเป็น

B3 = 3.1

เช็ค `i < nums.length` ถ้า true จะเข้าสู่การทำงาน แต่ถ้า false จะออกจากลูปแล้ว return count

B4 = 3.2

เช็ค `nums[i] == prev` ถ้า true จะทำการเช็คเงื่อนไขถัดไปที่ 3.3 แต่ถ้า false จะข้ามไปเช็ค 3.5 เลย

B5 = 3.3

เช็ค `!inClump` ถ้า true จะทำการ `inClump = true` และ `count + 1` แต่ถ้า false จะข้ามไปเช็ค 3.5 เลย

B6 = 3.5

เช็ค `nums[i] == prev` ถ้า true จะทำการกำหนด `prev = nums[i]` และ `inClump = false` แต่ถ้า false จะเป็นการจบลูปการทำงานรอบนั้นและ `i + 1`

Condition:

1.1 `nums == null`

1.2 `nums.length`

3.1 `i < nums.length`

3.2 `nums[i] == prev`

3.3 `!inClump`

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

3.5 nums[i] != prev

แบบฝึกปฏิบัติที่ 7.2 LINE COVERAGE

1. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1
ให้ออกแบบกรณีทดสอบเพื่อให้ได้ Line coverage = 100%
2. เขียนกรณีทดสอบที่ได้ พร้อมระบุบรรทัดที่ถูกตรวจสอบทั้งหมด
3. แสดงวิธีการคำนวณค่า Line coverage

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Path and Branch
TC01	null	0	Line No.:5,6,7
TC02	[]	0	Line No.:5,6,7
TC03	[1]	0	Line No.: 5,10-12,24
TC04	[1,2,3,3,4,4]	2	Line No.:5,10-24
TC05	[1,2,3,4,5,6]	0	Line No.: 5,10-24

Total Line = 19, Execution Line = 24 - 5 = 19

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

Line coverage = $(19 / 19) * 100 = 100\%$

แบบฝึกปฏิบัติที่ 7.3 BLOCK COVERAGE

1. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1
ให้ออกแบบกรณีทดสอบเพื่อให้ได้ Block coverage = 100%
2. เขียนกรณีทดสอบที่ได้ พร้อมระบุ Block ที่ถูกตรวจสอบทั้งหมด
3. แสดงวิธีการคำนวณค่า Block coverage

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Path and Branch
TC01	null	0	Block: 1.1, 1.3
TC02	[]	0	Block: 1.1, 1.2, 1.3
TC03	[1]	0	Block: 1.1, 1.2, 2, 4
TC04	[1,2,3,3,4,4]	2	Block: 1.1, 1.2, 2, 3.1,3.2, 3.3, 3.4, 3.5, 3.6, 3.7,4

Total Block = 12, Execution Block = 12

Block coverage = $(12 / 12) * 100 = 100\%$

แบบฝึกปฏิบัติที่ 7.3 BRANCH COVERAGE

4. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1
ให้ออกแบบกรณีทดสอบเพื่อให้ได้ Branch coverage = 100%
5. เขียนกรณีทดสอบที่ได้ พร้อมระบุ Path และ Branch ที่ถูกตรวจสอบทั้งหมด
6. แสดงวิธีการคำนวณค่า Branch coverage

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Path and Branch
TC01	null	0	Path: 1.1 -> 1.3

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

			Branch: B1
TC02	[]	0	Path: 1.1->1.2->1.3 Branch: B1, B2
TC03	[1,2,2,3]	1	Path: 1.1->1.2->1.3 Branch: B1,B2,B3,B4,B5,B6
TC04	[1,2,3,4]	0	Path: 1.1->1.2->2->3.1->3.2->3 .5->3.6->3.7->4 Branch: B1,B2,B3,B4,B5,B6

Branch coverage = $(6 / 6) * 100 = 100\%$

แบบฝึกปฏิบัติที่ 7.4 CONDITION COVERAGE

1. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1
ให้ออกแบบกรณีทดสอบเพื่อให้ได้ Condition coverage = 100%
2. เขียนกรณีทดสอบที่ได้ พร้อมระบุ Path และ Condition ที่ถูกตรวจสอบ
ทั้งหมด เช่น Condition A = T และ Condition B = F
3. แสดงวิธีการคำนวณค่า Condition coverage

ตอบ

Test Case No.	Input(s)	Expected Result(s)	Path and Condition
TC01	null	0	Path: 1.1->1.3

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

			Condition: 1.1 = True
TC02	[]	0	Path: 1.1->1.2->1.3 Condition: 1.1 = False, 1.2 = True
TC03	[1]	0	Path 1.1->1.2->2->3.1->4 Condition: 1.1 = False, 1.2 = False, 3.1= False
TC04	[1,2,3,4]	0	Path 1.1->1.2->2->3.1->3.2->3 .5->3.6->3.7->4 Condition: 1.1 = False, 1.2 = False, 3.1 = True (รอบสุดท้ายเป็น False), 3.2 = False, 3.5 = True

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

TC05	[1,1,2]	1	Path 1.1->1.2->2->3.1->3.2->3 .3->3.4->3.5->3.7->4 Condition 1.1=False,1.2=False,3.1 =True(รอบสุดท้าย false),3.2=True,3.3=True, 3.5=False
------	---------	---	--

Condition coverage = $(6 / 6) * 100 = 100\%$

แบบฝึกปฏิบัติที่ 7.5 BRANCH AND CONDITION COVERAGE (C/DC COVERAGE)

1. จาก Control Flow Graph (CFG) ของเมธอด countClumps() ในข้อที่ 1 ให้ออกแบบกรณีทดสอบให้ได้ C/DC coverage = 100%
2. เขียนกรณีทดสอบที่ได้ พร้อมระบุ Path, Branch, และ Condition ที่ถูกตรวจสอบทั้งหมด
3. แสดงวิธีการคำนวณค่า C/DC coverage
4. เขียนโค้ดสำหรับทดสอบตามกรณีทดสอบที่ออกแบบไว้ด้วย JUnit และบันทึกผลการทดสอบ

ตอบ

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

Test Case No.	Input(s)	Expected Result(s)	Actual Result(s)	Path, Branch, and Condition
TC01	null	0	Pass/Fail:Pass	Path 1.1->1.3 Branch: B1 Condition: 1.1=True
TC02	[]	0	Pass/Fail:Pass	Path 1.1->1.2->1.3 Branch: B1,B2 Condition: 1.1=False,1.2=Tru e
TC03	[1,2,3,3,4,4]	2	Pass/Fail:Pass	Path: 1.1->1.2->2->3.1->3.2->3.3->3.4->3.5->3.6->3.7->4 Branch: B1,B2,B3,B4,B5,B6 Condition: 1.1=False,1.2=False,3.1=True และ False,3.2=True

CP353201 Software Quality Assurance (1/2568)

ผศ.ดร.ชิตสุธา สุ่มเล็ก

Lab instruction

				และ False,3.3 =TrueและFalse, 3.5=Trueและ false
TC04	[1,2,3,4,5 ,6]	0	Pass/Fail:Pass	Path:1.1->1.2->2- >3.1->3.2->3.5->3 .6->3.7->4 Condition: 1.1=False,1.2=Fal se,3.1=True/False ,3.2=False,3.5=Tr ue

Total Branch = 6

Total Condition = 6

C/DC coverage = $(12/12) * 100 = 100\%$